attribute-oriented induction. **Concept description** is the most basic form of descriptive data mining. It describes a given set of task-relevant data in a concise and summarative manner, presenting interesting general properties of the data. Concept (or class) description consists of **characterization** and **comparison** (or **discrimination**). The former summarizes and describes a data collection, called the **target class**, whereas the latter summarizes and distinguishes one data collection, called the **target class**, from other data collection(s), collectively called the **contrasting class(es)**.

- **Concept characterization** can be implemented using **data cube (OLAP-based) approaches** and the **attribute-oriented induction approach**. These are attribute- or dimension-based generalization approaches. The **attribute-oriented induction approach** consists of the following techniques: *data focusing*, *data generalization by attribute removal or attribute generalization*, *count and aggregate value accumulation*, *attribute generalization control*, and *generalization data visualization*.
- **Concept comparison** can be performed using the attribute-oriented induction or data cube approaches in a manner similar to concept characterization. Generalized tuples from the target and contrasting classes can be quantitatively compared and contrasted.

4.7 Exercises

4.1 State why, for the integration of multiple heterogeneous information sources, many companies in industry prefer the *update-driven approach* (which constructs and uses data warehouses), rather than the *query-driven approach* (which applies wrappers and integrators). Describe situations where the query-driven approach is preferable to the update-driven approach.

4.2 Briefly compare the following concepts. You may use an example to explain your point(s).

(a) Snowflake schema, fact constellation, starnet query model

(b) Data cleaning, data transformation, refresh

(c) Discovery-driven cube, multifeature cube, virtual warehouse

4.3 Suppose that a data warehouse consists of the three dimensions *time, doctor*, and *patient*, and the two measures *count* and *charge*, where *charge* is the fee that a doctor charges a patient for a visit.

(a) Enumerate three classes of schemas that are popularly used for modeling data warehouses.

(b) Draw a schema diagram for the above data warehouse using one of the schema classes listed in (a).

(c) Starting with the base cuboid [*day*, *doctor*, *patient*], what specific *OLAP operations* should be performed in order to list the total fee collected by each doctor in 2010?

(d) To obtain the same list, write an SQL query assuming the data are stored in a relational database with the schema *fee* (*day, month, year, doctor, hospital, patient, count, charge*).

4.4 Suppose that a data warehouse for *Big_University* consists of the four dimensions *student, course, semester*, and *instructor*, and two measures *count* and *avg_grade*. At the lowest conceptual level (e.g., for a given student, course, semester, and instructor combination), the *avg_grade* measure stores the actual course grade of the student. At higher conceptual levels, *avg_grade* stores the average grade for the given combination.

(a) Draw a *snowflake schema* diagram for the data warehouse.

(b) Starting with the base cuboid [*student*, *course*, *semester*, *instructor*], what specific *OLAP operations* (e.g., roll-up from *semester* to *year*) should you perform in order to list the average grade of *CS* courses for each *Big_University* student.

(c) If each dimension has five levels (including all), such as "*student* < *major* < *status* < *university* < all", how many cuboids will this cube contain (including the base and apex cuboids)?

4.5 Suppose that a data warehouse consists of the four dimensions *date, spectator, location*, and *game*, and the two measures *count* and *charge*, where *charge* is the fare that a spectator pays when watching a game on a given date. Spectators may be students, adults, or seniors, with each category having its own charge rate.

(a) Draw a *star schema* diagram for the data warehouse.

(b) Starting with the base cuboid [*date*, *spectator*, *location*, *game*], what specific *OLAP operations* should you perform in order to list the total charge paid by student spectators at *GM_Place* in 2010?

(c) *Bitmap indexing* is useful in data warehousing. Taking this cube as an example, briefly discuss advantages and problems of using a bitmap index structure.

4.6 A data warehouse can be modeled by either a *star schema* or a *snowflake schema*. Briefly describe the similarities and the differences of the two models, and then analyze their advantages and disadvantages with regard to one another. Give your opinion of which might be more empirically useful and state the reasons behind your answer.

4.7 Design a data warehouse for a regional weather bureau. The weather bureau has about 1000 probes, which are scattered throughout various land and ocean locations in the region to collect basic weather data, including air pressure, temperature, and precipitation at each hour. All data are sent to the central station, which has collected such data for more than 10 years. Your design should facilitate efficient querying and online analytical processing, and derive general weather patterns in multidimensional space.

4.8 A popular data warehouse implementation is to construct a multidimensional database, known as a data cube. Unfortunately, this may often generate a huge, yet very sparse, multidimensional matrix.

(a) Present an example illustrating such a huge and sparse data cube.

(b) Design an implementation method that can elegantly overcome this sparse matrix problem. Note that you need to explain your data structures in detail and discuss the space needed, as well as how to retrieve data from your structures.

(c) Modify your design in (b) to handle *incremental data updates*. Give the reasoning behind your new design.

4.9 Regarding the *computation of measures* in a data cube:

(a) Enumerate three categories of measures, based on the kind of aggregate functions used in computing a data cube.

(b) For a data cube with the three dimensions *time, location*, and *item*, which category does the function *variance* belong to? Describe how to compute it if the cube is partitioned into many chunks.
Hint: The formula for computing *variance* is $\frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x_i})^2$, where $\bar{x_i}$ is the average of x_is.

(c) Suppose the function is "*top 10 sales*." Discuss how to efficiently compute this measure in a data cube.

4.10 Suppose a company wants to design a data warehouse to facilitate the analysis of moving vehicles in an online analytical processing manner. The company registers huge amounts of auto movement data in the format of (*Auto_ID, location, speed, time*). Each *Auto_ID* represents a vehicle associated with information (e.g., *vehicle_category, driver_category*), and each location may be associated with a street in a city. Assume that a street map is available for the city.

(a) Design such a data warehouse to facilitate effective online analytical processing in multidimensional space.

(b) The movement data may contain noise. Discuss how you would develop a method to automatically discover data records that were likely erroneously registered in the data repository.

(c) The movement data may be sparse. Discuss how you would develop a method that constructs a reliable data warehouse despite the sparsity of data.

(d) If you want to drive from A to B starting at a particular time, discuss how a system may use the data in this warehouse to work out a fast route.

4.11 Radio-frequency identification is commonly used to trace object movement and perform inventory control. An RFID reader can successfully read an RFID tag from a limited distance at any scheduled time. Suppose a company wants to design a data warehouse to facilitate the analysis of objects with RFID tags in an online analytical processing manner. The company registers huge amounts of RFID data in the format of *(RFID, at_location, time)*, and also has some information about the objects carrying the RFID tag, for example, *(RFID, product_name, product_category, producer, date_produced, price)*.

(a) Design a data warehouse to facilitate effective registration and online analytical processing of such data.

(b) The RFID data may contain lots of redundant information. Discuss a method that maximally reduces redundancy during data registration in the RFID data warehouse.

(c) The RFID data may contain lots of noise such as missing registration and misread IDs. Discuss a method that effectively cleans up the noisy data in the RFID data warehouse.

(d) You may want to perform online analytical processing to determine how many TV sets were shipped from the LA seaport to BestBuy in Champaign, IL, by *month*, *brand*, and *price_range*. Outline how this could be done efficiently if you were to store such RFID data in the warehouse.

(e) If a customer returns a jug of milk and complains that is has spoiled before its expiration date, discuss how you can investigate such a case in the warehouse to find out what the problem is, either in shipping or in storage.

4.12 In many applications, new data sets are incrementally added to the existing large data sets. Thus, an important consideration is whether a measure can be computed efficiently in an incremental manner. Use *count, standard deviation*, and *median* as examples to show that a distributive or algebraic measure facilitates efficient incremental computation, whereas a holistic measure does not.

4.13 Suppose that we need to record three measures in a data cube: min(), average(), and median(). Design an efficient computation and storage method for each measure given that the cube allows data to be *deleted incrementally* (i.e., in small portions at a time) from the cube.

4.14 In data warehouse technology, a multiple dimensional view can be implemented by a relational database technique (*ROLAP*), by a multidimensional database technique (*MOLAP*), or by a hybrid database technique (*HOLAP*).

(a) Briefly describe each implementation technique.

(b) For each technique, explain how each of the following functions may be implemented:

i. The generation of a data warehouse (including aggregation)

ii. Roll-up

iii. Drill-down

iv. Incremental updating

(c) Which implementation techniques do you prefer, and why?

4.15 Suppose that a data warehouse contains 20 dimensions, each with about five levels of granularity.

(a) Users are mainly interested in four particular dimensions, each having three frequently accessed levels for rolling up and drilling down. How would you design a data cube structure to support this preference efficiently?

(b) At times, a user may want to *drill through* the cube to the raw data for one or two particular dimensions. How would you support this feature?

4.16 A data cube, *C*, has *n* dimensions, and each dimension has exactly *p* distinct values in the base cuboid. Assume that there are no concept hierarchies associated with the dimensions.

(a) What is the *maximum number of cells* possible in the base cuboid?

(b) What is the *minimum number of cells* possible in the base cuboid?

(c) What is the *maximum number of cells* possible (including both base cells and aggregate cells) in the *C* data cube?

(d) What is the *minimum number of cells* possible in *C*?

4.17 What are the differences between the three main types of data warehouse usage: *information processing*, *analytical processing*, and *data mining*? Discuss the motivation behind *OLAP mining (OLAM)*.

4.8 Bibliographic Notes

There are a good number of introductory-level textbooks on data warehousing and OLAP technology—for example, Kimball, Ross, Thornthwaite, et al. [KRTM08]; Imhoff, Galemmo, and Geiger [IGG03]; and Inmon [Inm96]. Chaudhuri and Dayal [CD97] provide an early overview of data warehousing and OLAP technology. A set of research papers on materialized views and data warehouse implementations were collected in *Materialized Views: Techniques, Implementations, and Applications* by Gupta and Mumick [GM99].

The history of decision support systems can be traced back to the 1960s. However, the proposal to construct large data warehouses for multidimensional data analysis is credited to Codd [CCS93] who coined the term *OLAP* for *online analytical processing*. The OLAP Council was established in 1995. Widom [Wid95] identified several research problems in data warehousing. Kimball and Ross [KR02] provide an overview of the deficiencies of SQL regarding the ability to support comparisons that are common in the business world, and present a good set of application cases that require data warehousing and OLAP technology. For an overview of OLAP systems versus statistical databases, see Shoshani [Sho97].

Gray et al. [GCB+97] proposed the data cube as a relational aggregation operator generalizing group-by, crosstabs, and subtotals. Harinarayan, Rajaraman, and Ullman [HRU96] proposed a greedy algorithm for the partial materialization of cuboids in the computation of a data cube. Data cube computation methods have been investigated by numerous studies such as Sarawagi and Stonebraker [SS94]; Agarwal et al. [AAD+96]; Zhao, Deshpande, and Naughton [ZDN97]; Ross and Srivastava [RS97]; Beyer and Ramakrishnan [BR99]; Han, Pei, Dong, and Wang [HPDW01]; and Xin, Han, Li, and Wah [XHLW03]. These methods are discussed in depth in Chapter 5.

The concept of iceberg queries was first introduced in Fang, Shivakumar, Garcia-Molina et al. [FSGM+98]. The use of join indices to speed up relational query processing was proposed by Valduriez [Val87]. O'Neil and Graefe [OG95] proposed a bitmapped

join index method to speed up OLAP-based query processing. A discussion of the performance of bitmapping and other nontraditional index techniques is given in O'Neil and Quass [OQ97].

For work regarding the selection of materialized cuboids for efficient OLAP query processing, see, for example, Chaudhuri and Dayal [CD97]; Harinarayan, Rajaraman, and Ullman [HRU96]; and Sristava et al. [SDJL96]. Methods for cube size estimation can be found in Deshpande et al. [DNR+97], Ross and Srivastava [RS97], and Beyer and Ramakrishnan [BR99]. Agrawal, Gupta, and Sarawagi [AGS97] proposed operations for modeling multidimensional databases. Methods for answering queries quickly by online aggregation are described in Hellerstein, Haas, and Wang [HHW97] and Hellerstein et al. [HAC+99]. Techniques for estimating the top N queries are proposed in Carey and Kossman [CK98] and Donjerkovic and Ramakrishnan [DR99]. Further studies on intelligent OLAP and discovery-driven exploration of data cubes are presented in the bibliographic notes in Chapter 5.

5 Data Cube Technology

Data warehouse systems provide online analytical processing (OLAP) tools for interactive analysis of multidimensional data at varied granularity levels. OLAP tools typically use the *data cube* and a multidimensional data model to provide flexible access to summarized data. For example, a data cube can store precomputed measures (like count() and total_sales()) for multiple combinations of data dimensions (like *item*, *region*, and *customer*). Users can pose OLAP queries on the data. They can also interactively explore the data in a multidimensional way through OLAP operations like *drill-down* (to see more specialized data such as total sales per city) or *roll-up* (to see the data at a more generalized level such as total sales per country).

Although the data cube concept was originally intended for OLAP, it is also useful for data mining. **Multidimensional data mining** is an approach to data mining that integrates OLAP-based data analysis with knowledge discovery techniques. It is also known as *exploratory multidimensional data mining* and *online analytical mining* (*OLAM*). It searches for interesting patterns by exploring the data in multidimensional space. This gives users the freedom to dynamically focus on any subset of interesting dimensions. Users can interactively drill down or roll up to varying abstraction levels to find classification models, clusters, predictive rules, and outliers.

This chapter focuses on data cube technology. In particular, we study methods for data cube computation and methods for multidimensional data analysis. Precomputing a data cube (or parts of a data cube) allows for fast accessing of summarized data. Given the high dimensionality of most data, multidimensional analysis can run into performance bottlenecks. Therefore, it is important to study data cube computation techniques. Luckily, data cube technology provides many effective and scalable methods for cube computation. Studying these methods will also help in our understanding and further development of scalable methods for other data mining tasks such as the discovery of frequent patterns (Chapters 6 and 7).

We begin in Section 5.1 with preliminary concepts for cube computation. These summarize the data cube notion as a lattice of cuboids, and describe basic forms of cube materialization. General strategies for cube computation are given. Section 5.2 follows with an in-depth look at specific methods for data cube computation. We study both *full materialization* (i.e., where all the cuboids representing a data cube are precomputed

Data Mining: Concepts and Techniques. DOI: 10.1016/B978-0-12-381479-1.00005-8
© 2012 Elsevier Inc. All rights reserved.